DUST Jupyter guide

Fredrik Falk
freddo@ludd.ltu.se

Contents

1 Introduction

DUST or *`LUDD` Distributed Systems and Technologies* is a `LUDD` service to host projects and resources for all students, associations, and others connected to LTU. It provides a self-service interface into `LUDD`s to access server clusters and resources. By default, all students get access to 2 CPU cores, 2GB of RAM, and 2GB of network storage. Projects and courses may request additional resources if need arrises.

This guide will walk through setting up a Jupyter Notebook instance on DUST.

2 Getting started

First, sign in to https://dust.ludd.ltu.se, read through the DUST terms of service, and then accept them. Take note of what you're not allowed to use DUST for, and the stated limitations.

Accessing the dashboard you should see one personal project automatically generated for you. If you have a secret key to create a shared project, you may enter that and a shared project will be created.

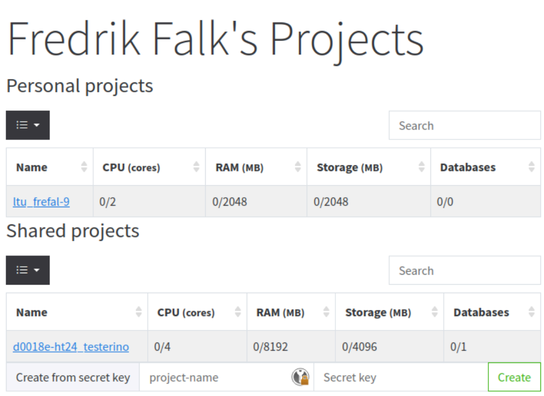

Figure 1: Project Dashboard

Select the project you wish to use.

3 Configuring storage

Enter the `Storage` tab and press `Create Storage Volume`. Grant it an appropriate amount of storage and give it an appropriate name.

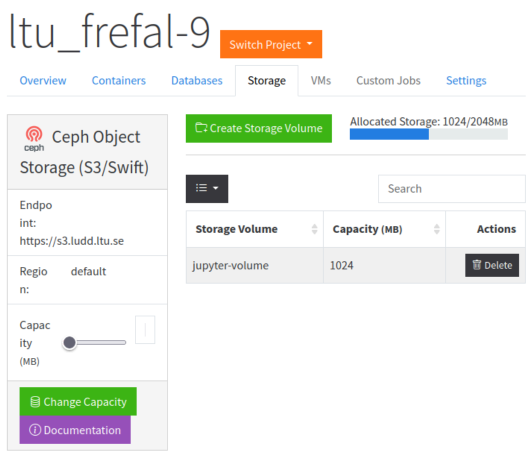

Figure 2: Storage tab with one volume created

Unfortunately, these volumes are created with file ownership for the `root` (id=0, gid=0) user, but Jupyter Notebook expects it to be owned by the `jovyan` (id=1000, gid=100) user, which must be rectified before we start the jupyter notebook.

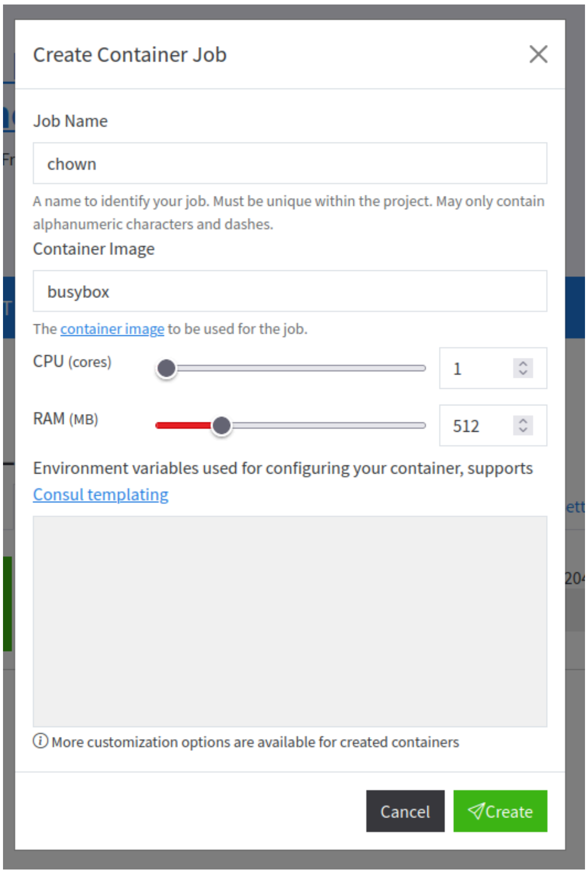

Figure 3: Create Container

Switch to the `Container` tab, and press `Create Container Job`. Set `Container Image` to `busybox`, which is a very lightweight container image with a set of system utilities, amongst them changing file ownership. See Figure 3.

Once the container is created, enter the configuration window for it. Here you'll need to set the following parameters:

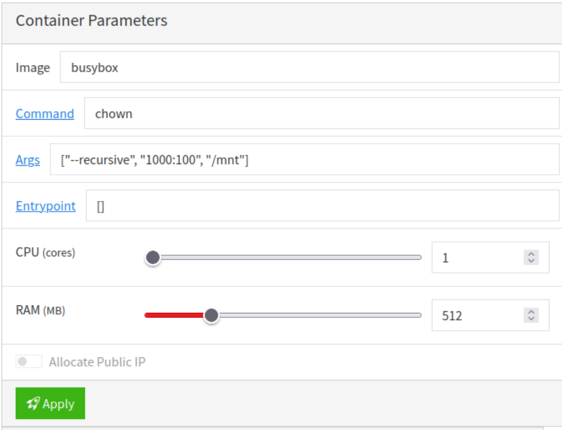

Figure 4: chown Container Parameters

- **Container Parameters**
  - **Command:** `chown`
  - **Args:** `["--recursive", "--verbose", "1000:100", "/mnt"]`
  - See Figure 4

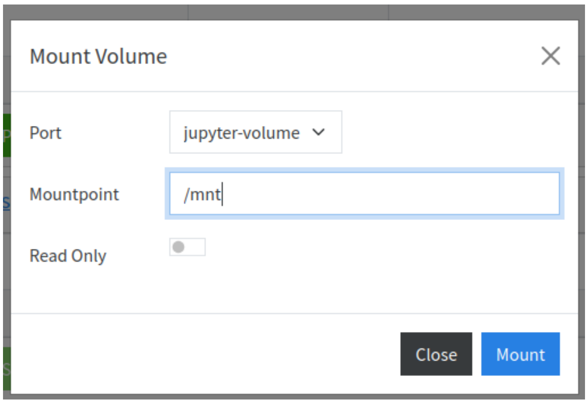

Figure 5: Mount Volume modal

- **Storage Volume Mounts**
  - **Volume:** The volume name you set, see Figure 5
  - **Mountpoint:** /mnt

Lastly, press `Apply`. This will launch the container job to change the storage volume ownership. The DUST frontend does not provide much information regarding how the job is progressing, for that we will use the Nomad interface, accessible by pressing `Open Nomad`. This should open the dashboard visible in Figure 6.

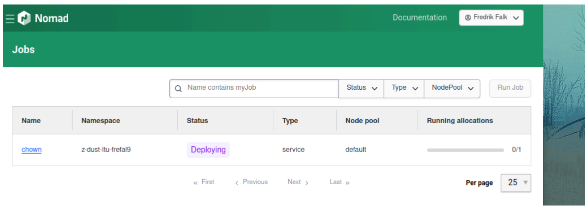

Figure 6: Nomad dashboard

Open the listed job, and you should see the current status and deployment history. The job will be in a restart loop as it is not meant to run one-off jobs, but that is fine in this case. Scrolling further down, you should see `Recent Allocations`. Select the topmost one and press `View Logs`. The logs should print `changed ownership of '/mnt' to 1000:100` as in Figure 7.

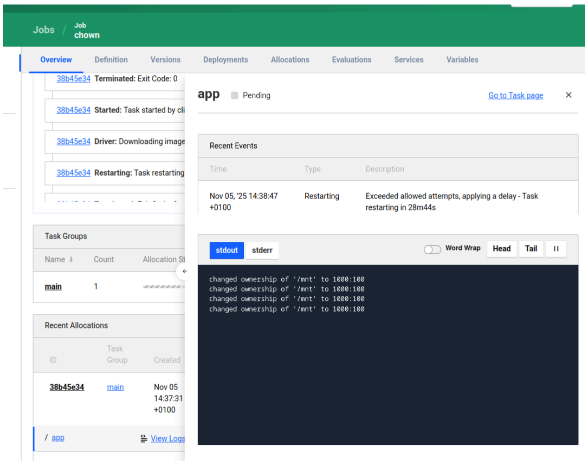

Figure 7: chown job logs

The file permissions should now be set correctly. Returning to the DUST dashboard, delete the container job you created and move on to the next part.

4 Setting up Jupyter Notebook

This is based on the Docker Jupyter guide[1].

First, return to the `Container Job`. Press `Create Container Job`. Set a descriptive name and `Container Image` to `quay.io/jupyter/minimal-notebook`. See Figure 8. Additionally, Jupyter provides several different images, each tailored to a different task.[2] In particular GPU workloads tend

[1] https://docs.docker.com/guides/jupyter/

[2] List of docker images https://jupyter-docker-stacks.readthedocs.io/en/latest/using/selecting.html

to require either an image containing pytorch or tensorflow.

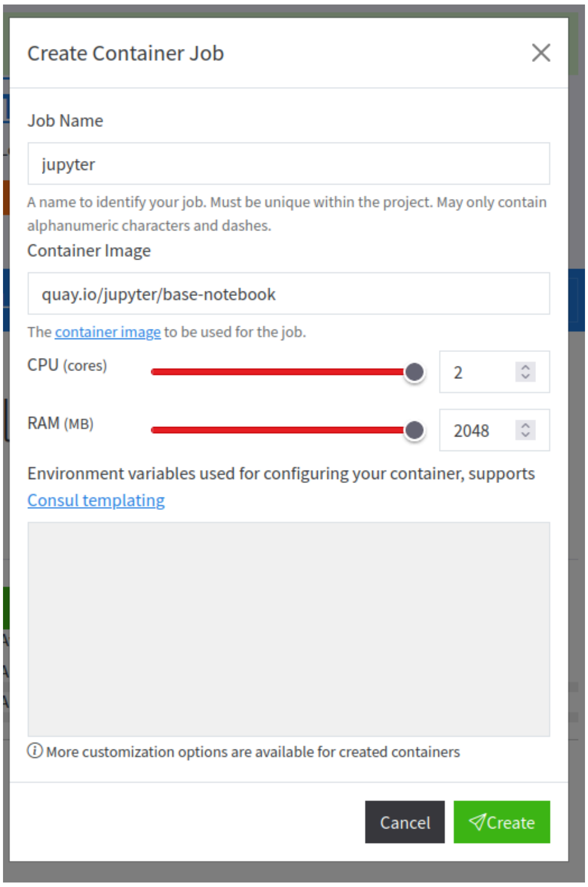

Figure 8: Jupyter job creation

Compared to the previous chown job, this will require more configuration.

- **Container Parameters:**
  - **Image:** `quay.io/jupyter/minimal-notebook`
  - **Args:** `["--NotebookApp.token='MY_SECRET_KEY'"]`
  - **Entrypoint:** `["start-notebook.py"]`
- **Ports:**
  - **1st port**
    - **Label:** `http`
    - **To:** 8888
- **Services:**
  - **1st service**
    - **Port:** http
    - **Domain:** `SOMETHING.dust.ludd.ltu.se`
- **Storage Volume Mounts:**
  - **1st mount**
    - **Volume:** The volume name you set, see Figure 5.
    - **Destination:** `/home/jovyan/work`
    - **Read Only:** false

See Figure 9 for complete configuration. Once these are all set, press `apply`. Once the job is running, which you can check in *Nomad*, the jupyter notebook should be accessible from https://SOMETHING.dust.ludd.ltu.se.

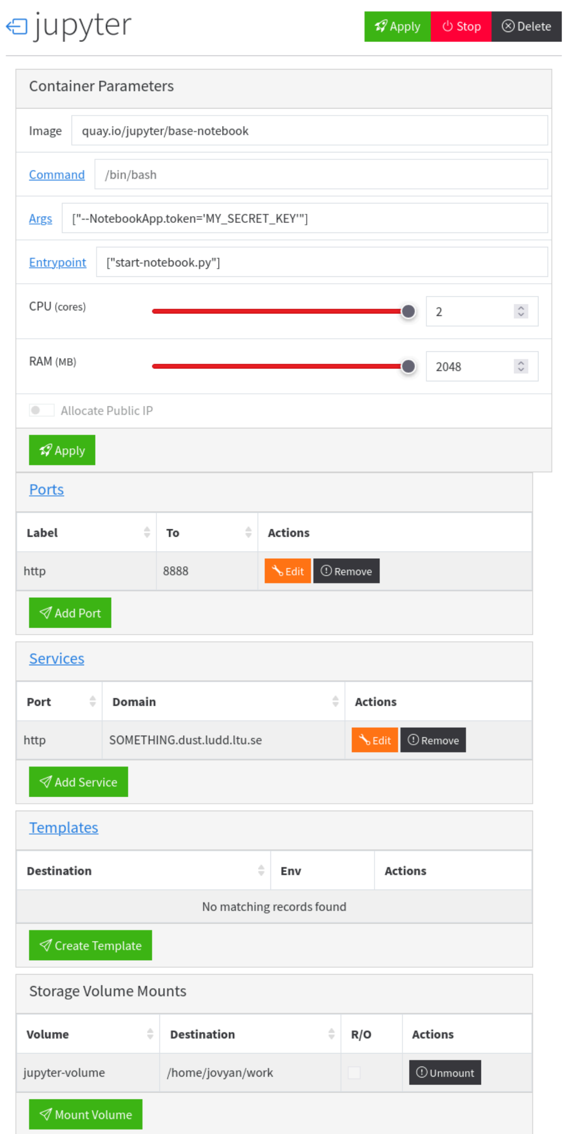

Figure 9: Complete container configuration

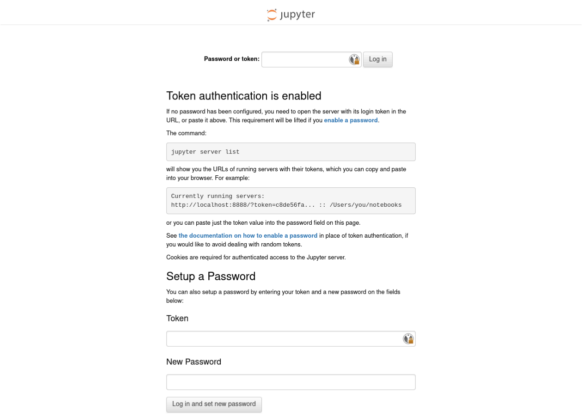

Figure 10: Jupyter login page

Once you've reached the login page as seen in Figure 10, enter the code you set as argument to the jupyter container in Figure 9. You should now have access to the notebook as seen in Figure 11!

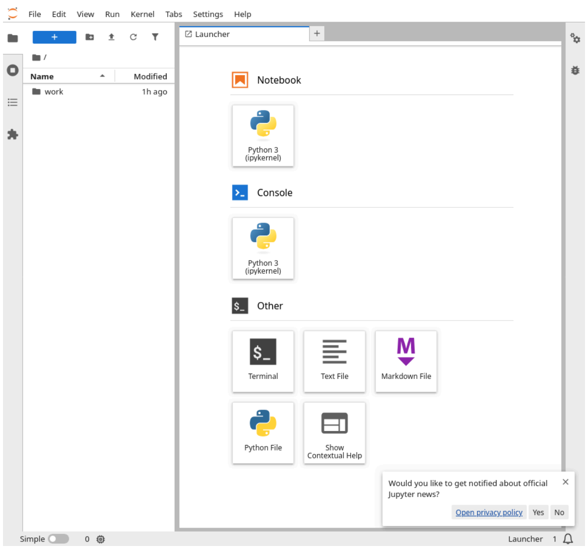

Figure 11: Running notebook

5 Limitations of DUST and Docker

Compared to running jupyter notebook locally, there are some things you need to keep in mind when running on DUST, or in docker. The single most critical point being that anything that is not placed in a storage volume will not be persisted across restarts. Following this guide, only the `/home/jovyan/work` directory will be persisted in network storage. We recommend that you test stop/starting the container job with test data to ensure that data isn't lost in case of restarts before conducting active work.

Restarts may occur for a variety of reasons, varying from power outages, the underlying server going down for maintenance, to a job of higher priority taking precedence and moving your job to a different server. Restarts are fairly uncommon and will typically be resolved in fairly short order, with your notebook coming back online within the span of a few minutes. You can check the current status and re-deployment logs in Nomad.

If the data stored in your notebook is of high value or cannot easily be recreated, we recommend that you take periodic external backups to ensure that data isn't lost. The storage solutions used by LUDD are resilient, but not infallible.

Lastly, if you need help or support, feel free to reach out to us at adm@ludd.ltu.se, our chat channels listed on https://ludd.ltu.se, or at our association room in A1401.